

function I

2024年9月21日 20:01

definition of a func:<return type><function-name>(<formal-parameter-list>)
{<function-body>}

It is legal to have an empty function body!

*{}在function中不可省略

*return type can be any built-in data types except **array**
const is OK!

*void means nothing->can be used as a return type

*modular/top-down programming: program->modules->function

*C++ supports 2 ways to pass arguments to a function:

1 pass-by-value (PBV), or call-by-value (CBV)

2 pass-by-reference (PBR), or call-by-reference (CBR)

*reference variable <type>&<variable>=<variable2>;

A reference variable is an alias of another variable

初始化时声明绑定变量后续不可改变，可以通过修改&bounded的任意变量来修改所有绑定值*在同一地址中*

return < expression >

The value of < expression > in the return statement should have the same type as the . Or, if it can be converted to the by coercion, otherwise it will be a compilation error.

If a function has a return value, the function body must have at least one return statement.

- A function that does not take any arguments from the caller may

- leave the <formal-parameter-list> empty.

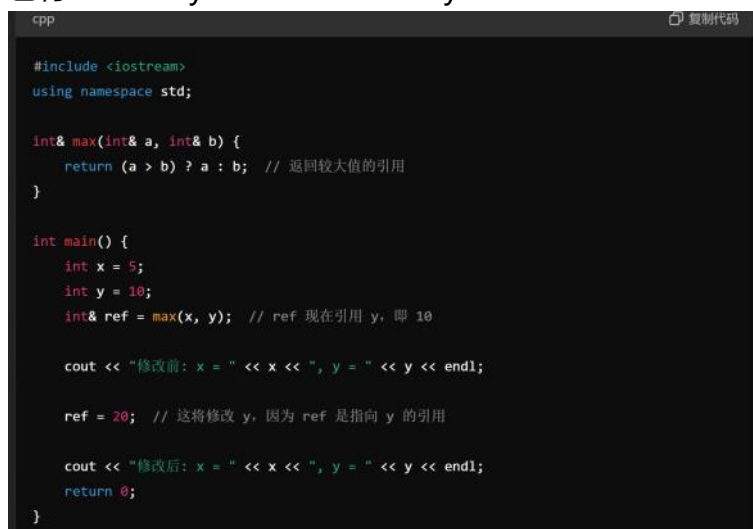
```
int fcn_example() { ... }
```

- put the <formal-parameter-list> as void.

```
void print_hkust(void) { cout << "hkust" << endl; }
```

有pass by value & pass by reference

也有 return by value & return by reference!!



```
cpp 复制代码

#include <iostream>
using namespace std;

int& max(int& a, int& b) {
    return (a > b) ? a : b; // 返回较大值的引用
}

int main() {
    int x = 5;
    int y = 10;
    int& ref = max(x, y); // ref 现在引用 y, 即 10

    cout << "修改前: x = " << x << ", y = " << y << endl;

    ref = 20; // 这将修改 y, 因为 ref 是指向 y 的引用

    cout << "修改后: x = " << x << ", y = " << y << endl;
    return 0;
}
```

通过修改ref 间接修改y

引用返回：在func中返回的必须是有效的左值** 例如某个变量或者常数

不能返回右值->暂时存在的一次性值，没有持久的储存位置，例如3*a

不能返回数组->数组本身不是左值是指向数组第一位的指针

**值返回：可以返回右值，因为是值返回所以创建了副本，会被存在在调用者的上下文中

！ A调用函数B，B需要先于A declared！ =defined！ 只要有defined就行，可以在A后再

define，declare方法：int b(){}

define只能define一次，

特殊情况：函数过载 （） 参数类型不同/数量不同/顺序不同

返回类型不能用于判断函数是否过载！！

比较函数是否相同时，编译器只会考虑以下几个方面：

1. **函数名**：必须相同。
2. **参数类型**：参数的类型、个数和顺序必须不同。
3. **const 限定符**：如果是成员函数，const 限定符的不同可以构成重载。